AI CRM Brain

Master Execution Roadmap

MVP v1.0 · 6-Sprint Plan · Deals-Only Scope

Architecture	View Layer	Database	Sprint
MVC Pattern	**Streamlit**	**PostgreSQL 16**	**1 of 6 Complete**

Document Purpose

This document is the single authoritative reference for the AI CRM Brain MVP v1.0 engineering effort. It reflects the state of the project after Sprint 1 completion and defines every remaining deliverable through Sprint 6.

All tasks are time-boxed, actionable, and scoped to the Deals module exclusively.

1. 6-Sprint Execution Roadmap

Sprint 1 · Foundation & API Integration | *Week 1*

STATUS: COMPLETE · OAuth, MVC Structure, Schema Extraction

Completed Deliverables

Task	Status
MVC directory structure initialized (models/, views/, controllers/, tests/)	✓ DONE
Python virtual environment, requirements.txt, .env pattern established	✓ DONE
Zoho OAuth 2.0 Authorization Code flow — get_new_access_token()	✓ DONE
Deals module JSON schema extracted via GET /crm/v3/Deals	✓ DONE
fetch_deals_schema() — field selection and response validation	✓ DONE
flatten_deals_to_csv() — nested JSON-to-flat-structure transformation	✓ DONE
real_sample_deals.json — local schema snapshot committed	✓ DONE
PostgreSQL 3-table DDL designed (zoho_deals, ml_predictions, llm_recommendations)	✓ DONE

Sprint 2 · Database Build & Full Ingestion Pipeline | *Week 2*

Theme: Zoho API → PostgreSQL — Production-Ready Data Layer

Goals

Deploy the PostgreSQL database and build a reliable, idempotent ingestion service that moves all active deal data from Zoho into the zoho_deals table cleanly and repeatably.

Task 2.1 — Deploy PostgreSQL + pgAdmin 4

- Create docker-compose.yml with postgres:16 service and dpage/pgadmin4 sidecar
- Load all environment variables (POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB) from .env
- Confirm pgAdmin 4 is accessible at localhost:5050 and can browse the database container

Task 2.2 — Run Alembic Migrations

- Initialize Alembic: alembic init alembic
- Write migration revision implementing all three DDL definitions from Section 2 of this document
- Execute alembic upgrade head — verify all 3 tables, indexes, and triggers appear in pgAdmin 4

Task 2.3 — Build ZohoCRMClient Service Class

- Refactor zoho_api.py into a ZohoCRMClient class (models/data_ingestion/zoho_client.py)
- Method: get_open_deals(page, per_page) — paginated fetch of active deals only
- Method: get_all_historical_deals() — full pull for initial training corpus (includes Closed Won/Lost)
- Implement automatic token refresh: every method call checks token expiry before executing
- Honour X-RATELIMIT-REMAINING response header — implement exponential backoff on 429 responses

Task 2.4 — Build DealTransformer Service

- Implement transform_deal(raw: dict) -> ZohoDealSchema (Pydantic model)
- Flatten nested objects: Account_Name.name, Contact_Name.name, Owner.name
- Compute amount_log = np.log1p(amount) at transform time
- Map Stage label to stage_encoded integer (ordinal 1–6) using a config dictionary
- Set is_closed_won: True for "Closed Won", False for "Closed Lost", None for all open stages

Task 2.5 — Data Validation Layer (V-1 / V-2 / V-3)

- V-1 Null Check: reject records where Amount, Stage, or Closing_Date IS NULL
- V-2 Type Assertion: Amount must be numeric; Closing_Date must parse as valid ISO 8601 date; Probability in 0–100
- V-3 Date Sanity: flag Closing_Date more than 2 years in the past or 5 years in the future
- Route all failed records to flagged_records table (zoho_deal_id, failure_stage, reason, raw_payload, flagged_at)

Task 2.6 — Data Quality Report Script

- Write scripts/data_quality_report.py — runs post-ingestion
- Outputs JSON to logs/quality_reports/: { total_fetched, passed, failed_v1/v2/v3, top_failure_fields, duration_seconds }
- Print formatted summary to stdout for CI/CD visibility

✓ **Sprint Deliverable**
A fully operational, idempotent ingestion pipeline running end-to-end.
python scripts/ingest.py triggers Zoho OAuth pull → Validation → Upsert into zoho_deals.
Minimum 20 real deal records visible and queryable in pgAdmin 4.
JSON quality report generated at logs/quality_reports/run_{timestamp}.json.

Sprint 3 · Feature Engineering & ML Model Training | *Week 3*

Theme: EDA → XGBoost Deal Closure Prediction Model

Goals

Train, evaluate, and register the core XGBoost Deal Closure Prediction model. Build the scikit-learn feature pipeline that transforms raw deal records into the fixed feature matrix the model expects at inference time.

Task 3.1 — Exploratory Data Analysis

- Load zoho_deals directly from PostgreSQL via pd.read_sql() into a Jupyter notebook
- Plot: amount distribution (log scale), stage frequency bar chart, probability vs is_closed_won scatter
- Audit class balance: ratio of True to False in is_closed_won — flag if imbalance exceeds 3:1
- Document key observations in notebooks/01_eda_deals.ipynb and commit to repository

Task 3.2 — Feature Engineering Pipeline

- Build scikit-learn Pipeline using ColumnTransformer (models/ml_engine/feature_pipeline.py)
- Numeric imputation: median strategy for amount, probability, expected_revenue
- Categorical imputation: constant "Unknown" for lead_source, then OneHotEncoder
- Ordinal passthrough: stage_encoded (already mapped; use as-is)
- Date features: days_to_close computed as (closing_date - today).days — impute mean for nulls
- CRITICAL: fit pipeline on training set only — serialise to models/ml_engine/artifacts/feature_pipeline.pkl

Source Column	Engineered Feature	Transformation	Imputation
amount	amount_log	np.log1p(amount)	Median fill
stage	stage_encoded	Ordinal map 1–6	No null expected
closing_date	days_to_close	(date - today).days	Mean fill
probability	probability_norm	value / 100	Median fill
expected_revenue	exp_rev_log	np.log1p(expected_revenue)	0 fill
lead_source	lead_source_ohe	OneHotEncoder (top N + Other)	Constant "Unknown"
is_closed_won	**target (y)**	**Binary 1/0 — NULL rows excluded**	**N/A — excluded**

Task 3.3 — Time-Based Train/Test Split

- Sort closed deals by closing_date ascending
- Train set: all deals closed before the 80th percentile closing_date cutoff
- Test set: all deals closed after the cutoff
- NEVER use random_state split on time-series sales data — this causes future data leakage

Task 3.4 — Model Training & Evaluation

- Primary: XGBoostClassifier with scale_pos_weight = n_negative / n_positive
- Baseline: RandomForestClassifier and LogisticRegression for sanity-checking
- Hyperparameter tuning: Optuna or GridSearchCV — time-box to 90 minutes maximum
- Metrics: AUC-ROC (primary), Precision-Recall curve, F1 at 0.5 threshold
- Log all runs to MLflow: hyperparameters, metrics, dataset hash, feature list, model artifact

Task 3.5 — SHAP Explainability

- Run shap.TreeExplainer on the best XGBoost model
- Generate SHAP summary plot — identify and document top 5 closure-driving features
- Persist SHAP values per deal to feature_vector JSONB column in ml_predictions — powers UI explanation panel

Task 3.6 — Model Registration & Inference Endpoint

- Register best model in MLflow Model Registry as deal_closure_predictor, version xgb_v1.0.0
- Serialise to models/ml_engine/artifacts/xgb_model.pkl
- Build predict_batch(deals_df) function — vectorised inference, returns base_probability per deal

✓ **Sprint Deliverable**
Trained, evaluated, and MLflow-registered XGBoost model achieving AUC-ROC > 0.70 on the held-out test set.
Serialised feature_pipeline.pkl — the fixed transformation applied at every inference call.
SHAP summary plot committed to docs/. Top 5 feature drivers documented.
predict_batch() function returning base_probability for a batch of deals.

Sprint 4 · LLM Integration & Active Batch Pull | *Week 4*

Theme: Few-Shot Prompting + APScheduler Scoring Pipeline

Goals

Wire the ML base score into the LLM recommendation engine using a few-shot prompting strategy. Build the Active Batch Pull scheduler that scores all open deals periodically and writes results to the database.

Task 4.1 — Few-Shot Prompt Engineering

- Define SYSTEM_PROMPT: role as "AI Sales Strategist", strict JSON output schema
- Curate 3–5 FEW_SHOT_EXAMPLES from real closed deals covering Won, Lost, and competitor-present scenarios

- Define strict output schema: { "adjusted_probability": float, "recommendation_ar": string }
- Validate LLM response against Pydantic model — retry with corrective prompt on parse failure (max 2 retries)
- Fallback: if LLM fails after retries, write adjusted_probability = base_probability and recommendation_ar = NULL

```
# models/ai_agents/recommender.py

SYSTEM_PROMPT = """
You are an expert AI Sales Strategist for a B2B CRM platform.
Evaluate the deal data below and respond ONLY with valid JSON:
{
  "adjusted_probability": <float, 0.00 to 1.00>,
  "recommendation_ar": "<2-3 sentence Arabic Next Best Action>"
}
Do not include any text outside the JSON object.
"""

FEW_SHOT_EXAMPLES = [
  {
    "input": {
      "base_probability": 0.82, "stage": "Negotiation",
      "days_to_close": 7,
      "custom_fields": {"Budget_Approved": "Yes", "Competitor": "None"}
    },
    "output": {
      "adjusted_probability": 0.91,
      "recommendation_ar": "الصفقة في مرحلة تفاوض والميزانية مؤكدة. تواصل مع صاحب القرار."
    }
  },
  # Add 2-4 more covering Closed-Lost and competitor-present scenarios
]
```

Task 4.2 — Data Fusion Package

- Build fuse_deal_payload(deal, base_probability) → dict
- Package: { base_probability, stage, days_to_close, amount, custom_fields (raw JSONB) }
- This fused package is the exact input sent to the LLM — also stored in llm_payload JSONB column for auditability

Task 4.3 — Active Batch Pull (APScheduler)

- Implement as APScheduler background task registered at FastAPI / controller startup
- Schedule: every 30 minutes — configurable via .env BATCH_INTERVAL_MINUTES
- Set max_instances=1 to prevent overlapping runs under slow network conditions
- Sequence per run: pull open deals → validate → score batch (XGBoost) → LLM per deal (asyncio.gather) → upsert predictions + recommendations
- Log batch metadata: batch_id UUID, deal count, error count, total duration in seconds

```
# controllers/batch_controller.py

@scheduler.scheduled_job("interval", minutes=30,
                         id="active_batch_pull",
                         max_instances=1)            # No overlapping runs
async def active_batch_pull():
```

```
    batch_id = uuid4()
    raw_deals    = await zoho_client.get_open_deals()
    validated    = validation_pipeline.run(raw_deals)
    predictions  = ml_engine.predict_batch(validated)  # Vectorised XGBoost
    await db.upsert_ml_predictions(predictions, batch_id)

    recs = await asyncio.gather(
        *[llm_service.generate(p) for p in predictions],
        return_exceptions=True   # Isolate per-deal LLM failures
    )
    await db.upsert_llm_recommendations(recs, batch_id)
    logger.info(f"Batch {batch_id}: {len(predictions)} deals scored.")
```

Task 4.4 — Dashboard API Endpoints

- GET /api/v1/deals/ranked — JOIN zoho_deals + latest llm_recommendations, ordered by adjusted_probability DESC
- GET /api/v1/deals/{deal_pk} — full deal detail: all 3 tables joined, including feature_vector SHAP data
- GET /api/v1/deals/summary — aggregate counts for dashboard KPI cards
- POST /api/v1/batch/trigger — manually fire an immediate batch pull cycle
- PATCH /api/v1/recommendations/{rec_pk}/action — write rep feedback (rep_action_taken, rep_feedback_at)

✓ **Sprint Deliverable**
End-to-end scoring pipeline operational: Zoho → XGBoost → Data Fusion → LLM → llm_recommendations.
APScheduler running on configurable interval with max_instances=1 guard.
All 5 API endpoints tested and returning correctly structured responses.
LLM fallback logic working — no batch failure cascades from single LLM errors.

Sprint 5 · Streamlit Dashboard — The View Layer | *Week 5*

Theme: North Star UI Build (see Section 4 for full spec)

Goals

Build the MVP v1.0 Streamlit dashboard that consumes the FastAPI endpoints and presents deal intelligence to the sales manager. The dashboard is the entire output surface — every previous sprint feeds into what the rep sees here.

Task 5.1 — Streamlit Project Scaffold

- Create views/dashboard.py as the main Streamlit entry point
- Install: streamlit, requests, pandas, plotly — add to requirements.txt
- Configure st.set_page_config: wide layout, page title "AI CRM Brain", favicon
- Load API_BASE_URL from .env — all data fetched via REST calls to the FastAPI backend

Task 5.2 — Header & KPI Cards

- Header row: product title, Arabic subtitle, last-synced timestamp, "Sync Now" button
- "Sync Now" calls POST /api/v1/batch/trigger and shows st.spinner until the batch completes

- Three st.metric cards: Total Active Deals, High Priority Count (score ≥ 75%), Average AI Score
- Color-code metric deltas using Streamlit's native delta parameter

Task 5.3 — Ranked Deals Table

- Fetch ranked list from GET /api/v1/deals/ranked
- Display as st.dataframe with column config: Tier badge (color-coded), Progress bars for ML Score and AI Score
- Add score delta column: ▲+N (green) or ▼-N (red) showing LLM adjustment
- Filter sidebar: priority tier (All / HIGH / MEDIUM / LOW), minimum score slider
- Clicking a row updates st.session_state.selected_deal_pk — triggers the detail view below

Task 5.4 — Deal Detail Panel

- Render below the table when st.session_state.selected_deal_pk is set
- Three columns: (1) Deal metadata, (2) Probability KPI cards, (3) Actions
- Arabic recommendation box: st.container with RTL text styling via custom CSS (direction: rtl)
- SHAP factors expander: st.expander("Why this score?") — top 4 features with signed contributions
- Action buttons: "✅ Mark Actioned" → PATCH /api/v1/recommendations/{rec_pk}/action

Task 5.5 — Integration Testing

- Full end-to-end test: Zoho pull → DB → API → Streamlit renders ranked list correctly
- Performance test: batch scoring 500 deals must complete in under 5 seconds (XGBoost vectorised; LLM calls parallelised)
- User simulation: walk through all Streamlit interactions as a sales manager persona

✓ **Sprint Deliverable**

Fully integrated Streamlit dashboard consuming live FastAPI data.
Sales manager can view ranked deal list, click into any deal, and read an Arabic NL recommendation.
Sync Now button triggers a live batch pull and refreshes the UI without manual page reload.
All Streamlit interactions tested end-to-end. Loom walkthrough recorded.

Sprint 6 · Hardening, Demo Prep & Final Delivery | *Week 6*

Theme: Polish, Pitch, and Ship

Goals

Harden the system for the demo environment, produce all documentation deliverables, record the product demo, and deliver the final pitch-ready presentation.

Task 6.1 — System Hardening

- Add structlog structured logging across all services — every API call, batch run, and LLM request tagged with batch_id
- Implement global FastAPI exception handler — never expose Python tracebacks to the client
- Add GET /api/v1/health endpoint: checks DB connectivity and model artifact availability

- Rate-limit guard: honour X-RATELIMIT-REMAINING header; log warnings when < 20 credits remain
- Implement docker-compose up single-command startup: db + pgadmin + api + dashboard in one command

Task 6.2 — Documentation Suite

- README.md: complete local dev setup in under 5 commands
- docs/model_card_xgb_v1.md: training data description, evaluation metrics, known limitations, retraining cadence
- docs/prompt_engineering.md: few-shot template versioning log and example evolution rationale
- FastAPI OpenAPI docs auto-verified at /docs — all 5 endpoints documented with request/response schemas

Task 6.3 — Demo Recording (5–8 minutes)

- Scene 1: Open dashboard — show ranked deal list with priority tiers and score badges
- Scene 2: Click into a HIGH priority deal — show probability breakdown and Arabic recommendation
- Scene 3: Expand SHAP panel — explain why the score is high or low in plain language
- Scene 4: Click Sync Now — show live batch trigger and dashboard refresh
- Scene 5: Click Mark Actioned — show feedback loop completing

Task 6.4 — Business Case & Pitch Deck

- Business Case slide: model projected revenue uplift — current close rate vs +5% improvement on pipeline value
- Pitch deck (6–8 slides): Problem → Solution → Live Demo → Data Flow Architecture → ROI Model → v2.0 Roadmap
- Export pitch deck to PDF — include as docs/pitch_deck_v1.pdf in repository

✓ **Sprint Deliverable**
Demo-ready, hardened system. docker-compose up spins the full stack in one command.
Full documentation suite committed: README, model card, prompt log, API docs.
Recorded demo video (5–8 min) exported and linked in README.
Pitch deck PDF committed. Business case ROI model included.

2. Database Schema Design (DDL)

Three tables form the complete Model layer. Each table maps to exactly one stage in the data pipeline. The schema is intentionally minimal for the MVP — every column earns its place by either training the ML model, feeding the LLM, or rendering on the dashboard.

2.1 zoho_deals

Dual-purpose: historical training corpus (closed deals with is_closed_won populated) and live prediction queue (open deals with is_closed_won = NULL). The JSONB custom_fields column is the bridge between the strict ML feature set and the LLM's contextual reasoning.

```
CREATE TABLE zoho_deals (

    -- ── Identity ──────────────────────────────────────────
    deal_pk          UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    zoho_deal_id     VARCHAR(64)  UNIQUE NOT NULL,
                     -- Zoho native ID; used as upsert conflict key

    -- ── Standard Fields (fixed ML feature set) ────────────
    deal_name        TEXT,
    amount           NUMERIC(15,2),
    stage            VARCHAR(100),             -- Raw label from Zoho
    stage_encoded    SMALLINT,                 -- 1=Qualification … 6=Closed
    closing_date     DATE,
    zoho_probability NUMERIC(5,2),             -- Zoho native estimate 0-100
                     -- Retained: strong baseline predictor; NOT the model output
    expected_revenue NUMERIC(15,2),            -- Retained: correlated with deal size at risk
    lead_source      VARCHAR(100),

    -- ── Flattened Nested Objects ──────────────────────────
    account_name     TEXT,
    contact_name     TEXT,
    deal_owner       TEXT,

    -- ── ML Target Variable ────────────────────────────────
    is_closed_won    BOOLEAN      DEFAULT NULL,
    -- NULL  = open/active deal
    -- TRUE  = Closed Won   (training label 1)
    -- FALSE = Closed Lost  (training label 0)

    -- ── Pre-Computed Features (stored for inference speed) ─
    amount_log       NUMERIC(10,6),            -- log1p(amount) — computed in Python
    days_to_close    INT,                      -- (closing_date - CURRENT_DATE)

    -- ── Custom / Unseen Fields ────────────────────────────
    custom_fields    JSONB        DEFAULT '{}',
    -- NOT used in ML training (prevents matrix mismatch)
    -- Passed as-is to the LLM payload for contextual reasoning

    -- ── Audit ─────────────────────────────────────────────
    ingested_at      TIMESTAMPTZ  DEFAULT NOW(),
    updated_at       TIMESTAMPTZ  DEFAULT NOW(),
    data_source      VARCHAR(20)  DEFAULT 'zoho_api'
);

-- Indexes
CREATE INDEX idx_zd_stage         ON zoho_deals (stage);
CREATE INDEX idx_zd_closed        ON zoho_deals (is_closed_won);
CREATE INDEX idx_zd_closing_date  ON zoho_deals (closing_date);
CREATE INDEX idx_zd_custom        ON zoho_deals USING GIN (custom_fields);

-- Auto-update updated_at on any row modification
CREATE OR REPLACE FUNCTION trg_set_updated_at()
RETURNS TRIGGER AS $$ BEGIN NEW.updated_at = NOW(); RETURN NEW; END; $$ LANGUAGE plpgsql;

CREATE TRIGGER set_updated_at BEFORE UPDATE ON zoho_deals
    FOR EACH ROW EXECUTE FUNCTION trg_set_updated_at();
```

Why zoho_probability and expected_revenue are retained

zoho_probability is Zoho's own estimate of deal closure. It is a strong baseline predictor because it encodes the sales rep's subjective confidence. The XGBoost model learns when to trust it and when to override it.

expected_revenue = amount × (probability / 100). It is a composite signal that correlates deal size with the rep's assessed likelihood — a useful feature for the revenue forecasting capability in v2.0.

2.2 ml_predictions

One row per deal per batch scoring run. Stores the base_probability from XGBoost and the exact feature_vector used — enabling full auditability and powering the SHAP factor display in the UI.

```
CREATE TABLE ml_predictions (

    -- ── Identity ──────────────────────────────────────────
    prediction_pk    UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    deal_pk          UUID          NOT NULL REFERENCES zoho_deals(deal_pk) ON DELETE CASCADE,
    zoho_deal_id     VARCHAR(64)   NOT NULL,   -- Denormalised for JOIN-free lookups

    -- ── Model Output ──────────────────────────────────────
    base_probability NUMERIC(5,4)  NOT NULL,   -- XGBoost predict_proba (0.0000-1.0000)
    base_score_pct   NUMERIC(5,2)              -- base_probability × 100 for display
        GENERATED ALWAYS AS (ROUND(base_probability * 100, 2)) STORED,

    -- ── Feature Snapshot ──────────────────────────────────
    feature_vector   JSONB         NOT NULL,
    -- The exact feature dict fed to XGBoost for this prediction.
    -- Enables SHAP replay and model drift detection.
    -- Example: {"amount_log": 10.3, "stage_encoded": 4, "days_to_close": 12}

    -- ── Model Provenance ──────────────────────────────────
    model_version    VARCHAR(32)   NOT NULL,   -- e.g. "xgb_v1.0.0"
    model_run_id     VARCHAR(64),              -- MLflow run_id for experiment tracing

    -- ── Batch Metadata ────────────────────────────────────
    batch_id         UUID,
    scored_at        TIMESTAMPTZ   DEFAULT NOW()
);

CREATE INDEX idx_mlp_deal_pk    ON ml_predictions (deal_pk);
CREATE INDEX idx_mlp_batch_id   ON ml_predictions (batch_id);
CREATE INDEX idx_mlp_scored_at  ON ml_predictions (scored_at DESC);

-- One deal cannot be scored twice in the same batch
CREATE UNIQUE INDEX uq_mlp_deal_batch ON ml_predictions (deal_pk, batch_id);
```

2.3 llm_recommendations

The final output surface. Stores the LLM-adjusted score, the Arabic Next Best Action text, the computed priority_tier, and the rep feedback loop columns. This is the only table the Streamlit dashboard reads for ranking and rendering.

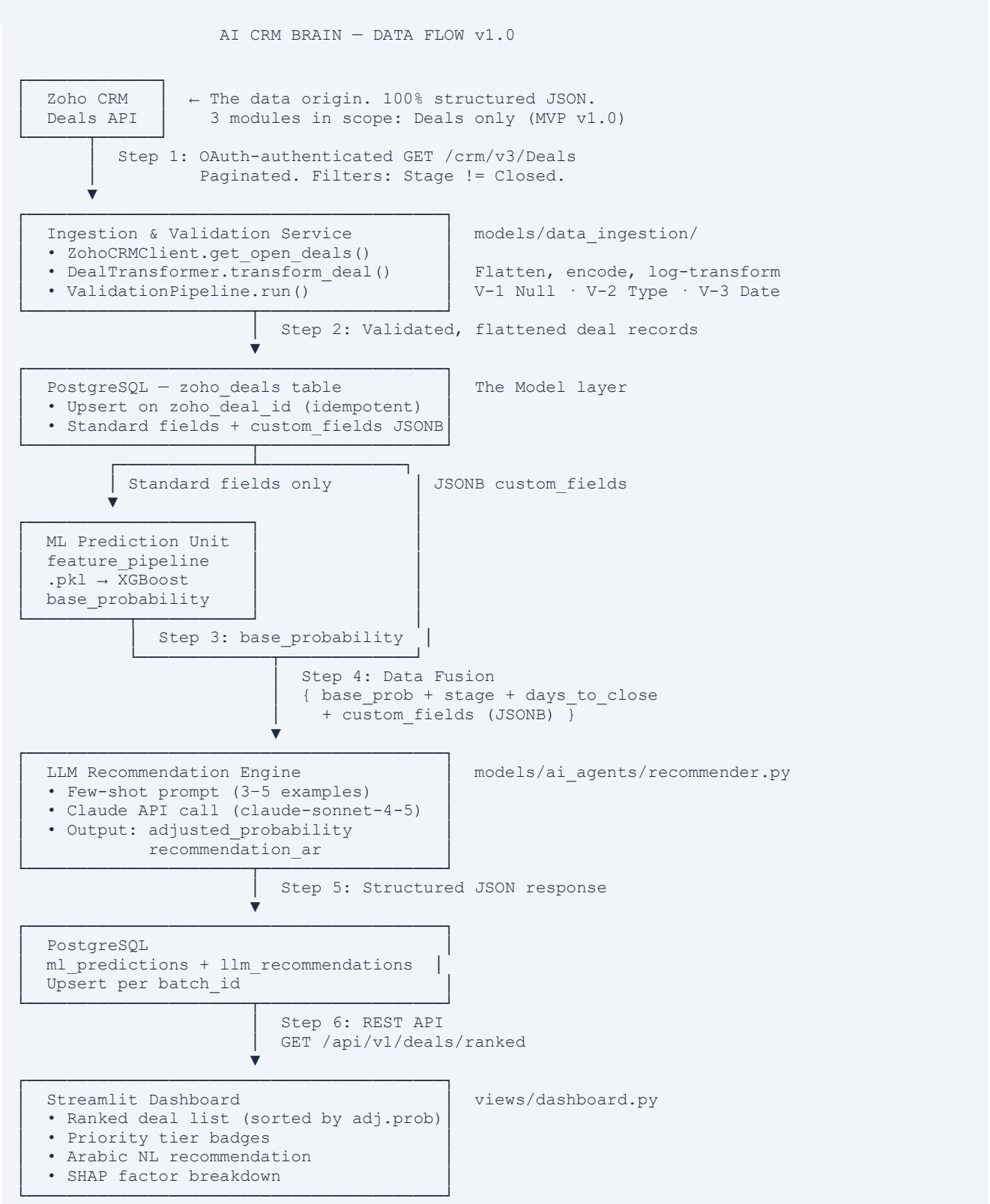
```
CREATE TABLE llm_recommendations (

    -- ── Identity ──────────────────────────────────────────
    recommendation_pk  UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    prediction_pk      UUID          NOT NULL REFERENCES ml_predictions(prediction_pk)
                                     ON DELETE CASCADE,
    deal_pk            UUID          NOT NULL REFERENCES zoho_deals(deal_pk)
                                     ON DELETE CASCADE,
    zoho_deal_id       VARCHAR(64)   NOT NULL,

    -- ── LLM Input (stored for debugging and prompt improvement) ──
    llm_payload        JSONB         NOT NULL,
    -- Exact JSON sent to Claude: { base_probability, stage, custom_fields, ... }

    -- ── LLM Output ──────────────────────────────────────────
    adjusted_probability NUMERIC(5,4) NOT NULL,
    adjusted_score_pct   NUMERIC(5,2)
        GENERATED ALWAYS AS (ROUND(adjusted_probability * 100, 2)) STORED,

    score_delta          NUMERIC(5,2)
        GENERATED ALWAYS AS (
            ROUND((adjusted_probability -
                   (llm_payload->>'base_probability')::NUMERIC) * 100, 2)
        ) STORED,
    -- Positive = LLM upgraded the ML score
    -- Negative = LLM downgraded (e.g. competitor present in custom fields)

    -- ── Natural Language Output ─────────────────────────────
    recommendation_ar  TEXT          NOT NULL,  -- Arabic Next Best Action
    recommendation_en  TEXT,                    -- Optional English mirror

    -- ── Priority Tier (computed for dashboard filtering) ──
    priority_tier      VARCHAR(10)
        GENERATED ALWAYS AS (
            CASE
                WHEN adjusted_probability >= 0.75 THEN 'HIGH'
                WHEN adjusted_probability >= 0.45 THEN 'MEDIUM'
                ELSE 'LOW'
            END
        ) STORED,

    -- ── LLM Provenance ──────────────────────────────────────
    llm_model_id       VARCHAR(64)   NOT NULL,  -- e.g. "claude-sonnet-4-5"
    prompt_version     VARCHAR(16)   NOT NULL,  -- e.g. "v1.3"
    llm_latency_ms     INT,

    -- ── Batch ───────────────────────────────────────────────
    batch_id           UUID,
    generated_at       TIMESTAMPTZ   DEFAULT NOW(),

    -- ── Rep Feedback Loop ───────────────────────────────────
    rep_action_taken   BOOLEAN       DEFAULT NULL,
    rep_feedback_at    TIMESTAMPTZ   DEFAULT NULL
);

CREATE INDEX idx_llmr_deal_pk      ON llm_recommendations (deal_pk);
CREATE INDEX idx_llmr_priority     ON llm_recommendations (priority_tier);
CREATE INDEX idx_llmr_generated_at ON llm_recommendations (generated_at DESC);

-- Dashboard primary query index: latest rec per deal, sorted by score
CREATE INDEX idx_llmr_dashboard
    ON llm_recommendations (deal_pk, generated_at DESC, adjusted_probability DESC);
```

3. Data Flow Architecture

A step-by-step trace of a single active deal moving through the complete AI CRM Brain pipeline — from its origin in Zoho CRM to appearing as a ranked, recommended card on the Streamlit dashboard.

3.1 Visual Pipeline Overview



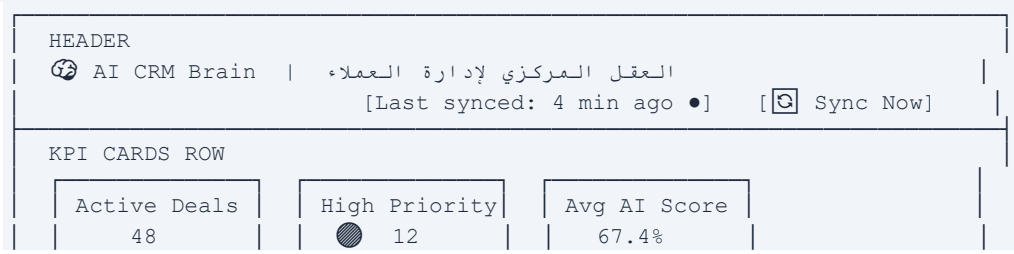
3.2 Step-by-Step Trace: Single Active Deal

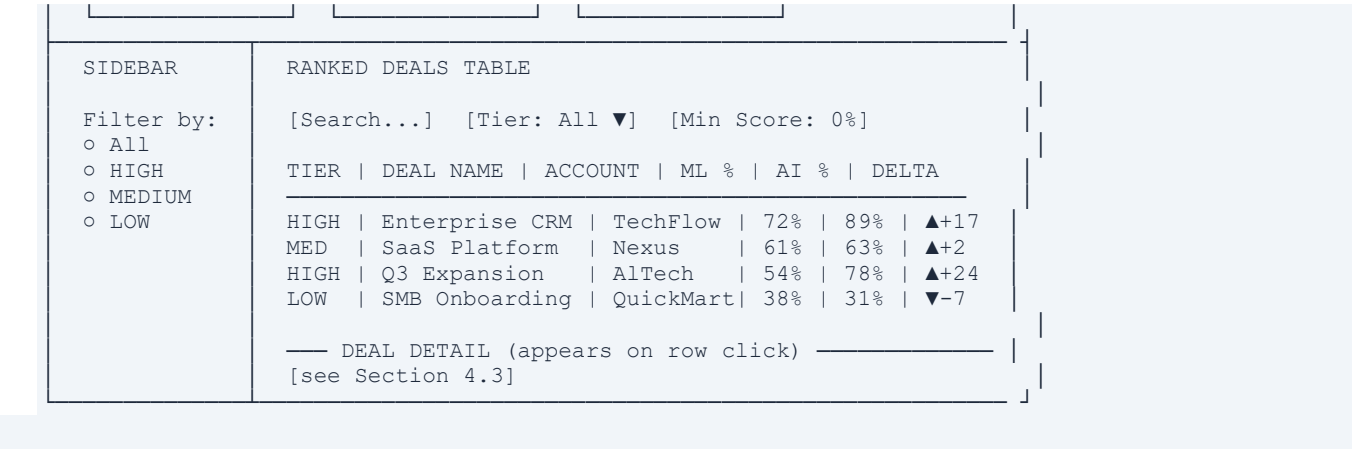
Step	Component	What Happens
1	Zoho API	APScheduler fires active_batch_pull(). ZohoCRMClient calls GET /crm/v3/Deals?fields=Amount,Stage,... with OAuth header. Response: JSON array of open deals.
2	Transformer	DealTransformer.transform_deal() flattens nested objects (Account_Name.name), maps Stage to stage_encoded (e.g. "Negotiation" → 4), computes amount_log = log1p(48000) = 10.78, sets is_closed_won = NULL.
3	Validation	ValidationPipeline checks: Amount not null ✓, Closing_Date parses as ISO date ✓, date is not 3 years past ✓. Record passes all 3 stages → written to zoho_deals via UPSERT.
4	ML Inference	feature_pipeline.pkl applies imputation and encoding to standard fields only. XGBoost.predict_proba() returns base_probability = 0.72. Written to ml_predictions with feature_vector JSONB snapshot.
5	Data Fusion	fuse_deal_payload() builds: { "base_probability": 0.72, "stage": "Negotiation", "days_to_close": 12, "custom_fields": {"Competitor": "Oracle", "Budget_Approved": "Yes"} }.
6	LLM Call	SYSTEM_PROMPT + FEW_SHOT_EXAMPLES + fused payload sent to Claude. Response: { "adjusted_probability": 0.81, "recommendation_ar": "وجود منافس رئيسي يستوجب التحرك الفوري..." }. Response validated via Pydantic.
7	DB Write	llm_recommendations row upserted: adjusted_probability=0.81, score_delta=+9 (0.81-0.72), priority_tier="HIGH" (computed column), recommendation_ar stored.
8	Dashboard	Streamlit polls GET /api/v1/deals/ranked every 30 seconds. Deal appears at rank #1 with HIGH badge, 81% AI Score, ▲+9 delta, and Arabic recommendation visible in the detail drawer.

4. Dashboard UI Vision — MVP v1.0

This section is the North Star specification for the Sprint 5 Streamlit build. Every component maps to a concrete API endpoint and a database column. The sales manager's complete experience is defined here so the engineering team has a fixed target throughout development.

4.1 Overall Layout





4.2 Component-by-Component Specification

Header Bar

- Left: Product logo icon + "AI CRM Brain" in bold + Arabic subtitle in lighter weight
- Center: Sync status dot — green ● if batch ran within 35 min, yellow ● if stale
- Center: "Last synced: N min ago" — computed from batch_id.generated_at
- Right: "🔄 Sync Now" button — calls POST /api/v1/batch/trigger, shows st.spinner, refreshes on complete

KPI Cards Row

Active Deals	High Priority	Avg AI Score
48	**12**	**67.4%**
COUNT WHERE is_closed_won IS NULL	priority_tier = "HIGH"	AVG(adjusted_score_pct)

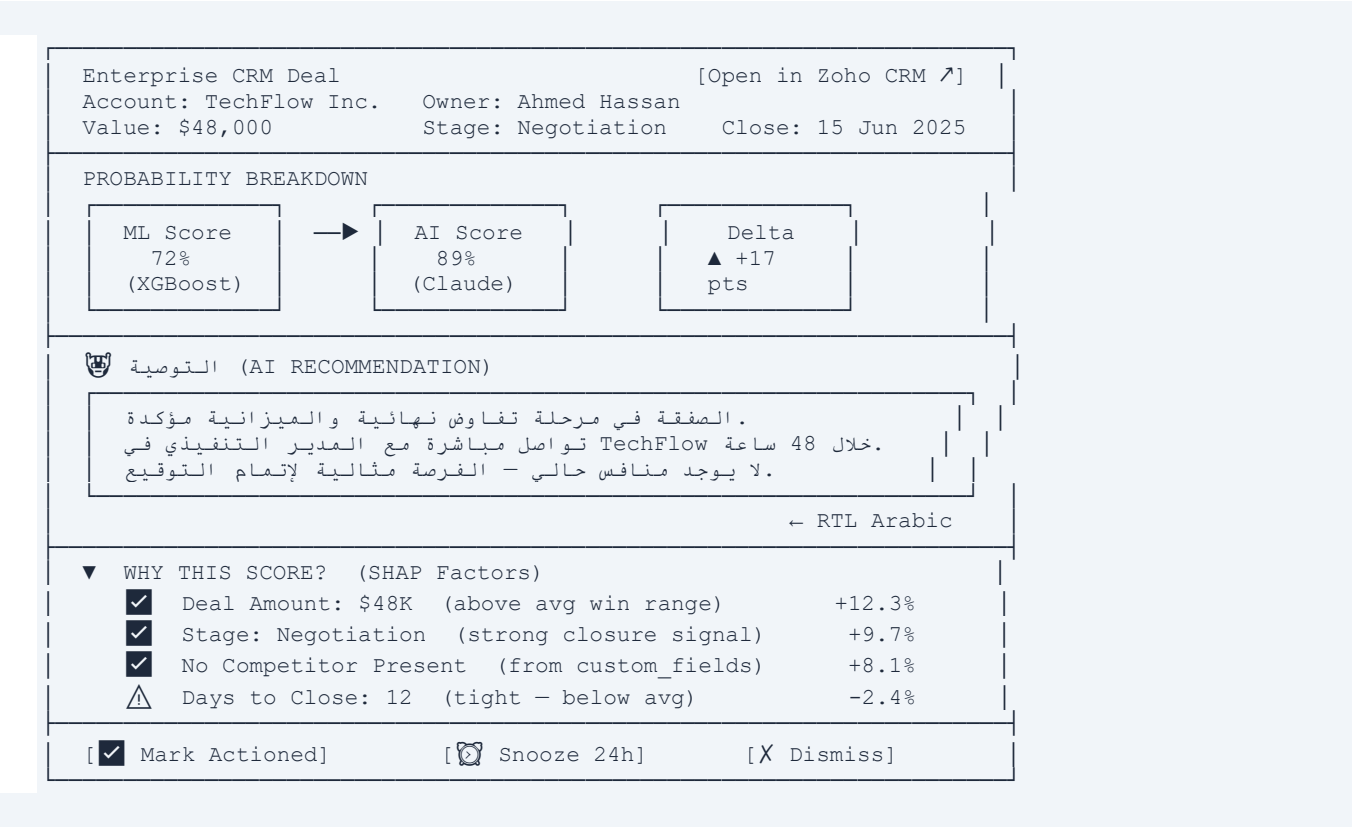
4.3 Ranked Deals Table — Column Specification

Column	Data Source	Render Style	HIGH	MEDIUM	LOW
TIER	priority_tier (computed)	Color pill badge	Red pill	Amber pill	Green pill
DEAL NAME	deal_name	Text, clickable row	—	—	—
ACCOUNT	account_name	Text	—	—	—
ML SCORE	base_score_pct	Blue progress bar + %	—	—	—
AI SCORE	adjusted_score_pct	Teal progress bar + %	—	—	—
DELTA	score_delta	▲+N green / ▼-N red	—	—	—

Column	Data Source	Render Style	HIGH	MEDIUM	LOW
CLOSE DATE	closing_date	Date, red if < 14 days	—	—	—

4.4 Deal Detail Panel — Slide-In View

Opens below the table (or as a right-side panel in wide layout) when the sales manager clicks any row. Never navigates away — the ranked list remains visible.



UI Element	Data Source	Streamlit Implementation
Deal header	zoho_deals: deal_name, account_name, amount, closing_date, stage	st.subheader + st.columns(3)
ML Score card	ml_predictions.base_score_pct	st.metric("ML Score", "72%")
AI Score card	llm_recommendations.adjusted_score_pct	st.metric("AI Score", "89%", delta="+17")
Delta card	llm_recommendations.score_delta	st.metric("Delta", "▲+17", delta_color="normal")
Arabic recommendation	llm_recommendations.recommendation_ar	st.markdown with custom CSS: direction:rtl
SHAP factors	ml_predictions.feature_vector (JSONB)	st.expander → st.dataframe of top 4 features

UI Element	Data Source	Streamlit Implementation
"Mark Actioned" btn	llm_recommendations.rep_action_taken	st.button → PATCH /api/v1/recommendations/{pk}/action

4.5 Streamlit RTL Arabic Styling

The Arabic recommendation box requires a custom CSS injection to render correctly in a left-to-right Streamlit layout:

```
# views/dashboard.py

RTL_STYLE = """
<style>
.arabic-recommendation {
    direction: rtl;
    text-align: right;
    font-family: "Noto Sans Arabic", "Arial", sans-serif;
    font-size: 1.05rem;
    line-height: 1.8;
    background-color: #CCFBF1;   /* lTeal */
    border-right: 4px solid #0D9488;
    padding: 16px 20px;
    border-radius: 6px;
    margin: 12px 0;
}
</style>
"""

st.markdown(RTL_STYLE, unsafe_allow_html=True)
st.markdown(
    f'<div class="arabic-recommendation">{recommendation_ar}</div>',
    unsafe_allow_html=True
)
```